GraphMana: graph-native data management for population genomics projects

Jianfeng Mao[1*]

[1*][Department to be completed], [Institution to be completed], [City], [Country].

Corresponding author(s). E-mail(s): jianfeng.mao@gmail.com;

Abstract

Population genomics projects rely on fragmented file-based workflows that lose provenance and require full reprocessing when samples are added. GraphMana is a data management platform built on graph database technology that stores variant data as packed genotype arrays with pre-computed population statistics, enabling incremental sample addition, provenance tracking, cohort management, and export to 17 formats from a single persistent source. Applied to the 1000 Genomes Project (3,202 samples, 70.7 million variants), GraphMana completes a multi-phase project lifecycle—import, incremental addition, annotation, cohort extraction, and multi-format export—otherwise requiring ad hoc scripting across multiple tools.

Keywords: population genomics, data management, graph database, variant call format, genotype storage, reproducibility

Declining sequencing costs and growing recognition that population-scale variation underlies both disease risk and adaptive potential have driven a rapid expansion of population genomics projects across biomedicine, agriculture, and biodiversity research. In human genetics, the UK Biobank has released whole-genome sequences for 490,640 participants, the NIH All of Us Research Program has surpassed 414,000 genomes with a goal of one million, and the TOPMed consortium has sequenced over 180,000 individuals for heart, lung, blood, and sleep disorder genetics. In crop science, the 3,000 Rice Genomes Project resequenced 3,010 accessions across 89 countries to catalog 29 million SNPs underlying cultivated rice diversity, while similar population-scale efforts are underway in wheat, maize, soybean, and other staple crops to accelerate genomic selection for yield and climate resilience. In livestock, the 1000

Bull Genomes Project has sequenced 2,703 cattle representing global breed diversity, with companion initiatives now extending to buffalo, sheep, and aquaculture species. In ecology, the 1001 Genomes Project catalogued standing variation across the natural range of *Arabidopsis thaliana*, and the Earth BioGenome Project has sequenced 3,000 eukaryotic species toward a goal of 1.67 million. These projects share a common trajectory: sample counts grow from hundreds to thousands as costs fall, new batches of sequencing data arrive throughout the project's lifetime, and the analytical scope expands from initial variant calling to population structure, selection scans, functional annotation, and multi-format data sharing.

Consider the daily reality of managing such a project. On Monday, a sequencing core delivers 200 new whole-genome samples; integrating them means regenerating every VCF [9], every PLINK binary [5], every TreeMix [12] input, every site frequency spectrum for dadi [10]—because file-based formats encode the complete sample set and cannot be extended in place. On Tuesday, a collaborator requests an EIGENSTRAT [13] file for AdmixTools, filtered to European samples with MAF above 5%; fulfilling the request requires a bespoke script whose parameters will not be recorded alongside the output it produces. On Wednesday, a new ClinVar release arrives and the functional annotations must be updated—but since annotations are embedded in the VCF header, updating means rewriting the entire file, even though the genotype data has not changed. On Thursday, the PI asks which samples were included in the allele frequency table shared with a collaborator last month, and what quality filters were applied; answering requires forensic reconstruction from directory timestamps and notebook entries. By Friday, the data manager has spent more time coordinating files, formats, and provenance than doing science. These are not computational bottlenecks—bcftools [6] can merge VCF files in minutes; PLINK can recode genotypes in seconds—but *coordination* bottlenecks that scale with project complexity rather than dataset size. This coordination cost falls in a gap between single-investigator projects (where manual tracking suffices) and biobank-scale programs (where platforms such as Hail [4] provide managed infrastructure). For the thousands of research groups running projects with hundreds to tens of thousands of samples—the scale at which most population genomics research actually occurs—no purpose-built data management solution exists.

The root cause of these coordination bottlenecks is that population genomics data is inherently relational—variants belong to chromosomes, samples belong to populations, genes carry functional consequences, annotations have versions—yet file-based formats flatten these relationships into disconnected tabular files. Several data management approaches could, in principle, address this problem (Table 1). Relational databases can store structured metadata but require schema-level JOIN operations to reconstruct variant–gene–population relationships and do not natively support extensible byte-array properties for packed genotypes. Columnar array stores (Zarr, HDF5) and distributed frameworks (Hail) excel at large-scale matrix computation but treat annotations, cohorts, and provenance as external metadata rather than co-located, independently updatable entities. Graph databases offer a complementary approach: data is stored as *nodes* (entities) connected by typed *edges* (relationships), and queries traverse these connections directly. This representation makes several project-lifecycle

operations—incremental sample addition, annotation versioning, cohort definition, and provenance binding—*persistent and local* to the data they describe, rather than requiring regeneration of the entire dataset when one relationship changes. We note that graph databases are not the only viable architecture; rather, the graph-native representation provides a particularly natural fit for the relationship-rich structure of population genomics projects.

Table 1 Comparison of data management architectures for population genomics. Each column evaluates whether the architecture supports the operation natively (•), with external scripting (○), or not at all (—).

Operation	Flat files	Relational DB	Columnar/Hail	GraphMana
Incremental sample addition	—	○	○	•
Annotation versioning	—	○	—	•
Provenance binding	—	○	○	•
Cohort as query	—	•	•	•
Multi-format export (17)	○	—	○	•
In-place annotation update	—	•	—	•
Regeneration avoidance	—	—	○	•

GraphMana exploits graph-native representation to replace the fragmented file collection with a persistent graph database that serves as the project's analytical record (Fig. 1a). The platform organizes 58 CLI commands into nine functional domains—data import, annotation management, data export, sample and cohort management, quality control, provenance tracking, database administration, status reporting, and infrastructure deployment (Supplementary Fig. 1)—providing a complete command-line interface for the data management lifecycle. All operations require no knowledge of Python, graph query languages, or database administration.

The graph schema (Fig. 1b) represents each biallelic variant as a Variant node connected by typed edges to Chromosome, Gene, Sample, and Population nodes. Variant nodes carry a packed genotype array (2 bits per sample, 4 samples per byte) and precomputed population-level allele count, sample count, and frequency arrays. Because these relationships are explicit edges in the graph, annotations can be updated by modifying edge properties without touching the genotype data, cohorts can be defined as graph traversal queries without extracting files, and provenance is recorded as graph nodes linked to the operations that produced them. This encoding achieves a 125-fold storage reduction compared with per-sample edge representations (Supplementary Table 3) and enables two access paths (Fig. 1c). The FAST PATH reads population arrays directly—constant time regardless of sample count—serving TreeMix [12], site frequency spectra (dadi [10], fastsimcoal2 [11]), BED, and allele frequency tables. The FULL PATH unpacks genotype arrays for per-sample formats (VCF, PLINK 1.9/2.0 [5], EIGENSTRAT [13], Beagle [14], STRUCTURE [15], Genepop, and others) in time linear in N.

In total, 17 export formats are produced from the database (Supplementary Note 2). Of these, 6 formats (VCF, PLINK 1.9, EIGENSTRAT, TreeMix, SFS-dadi,

SFS-fastsimcoal2) have been validated by successfully loading their output into the corresponding downstream tools (Supplementary Table 4). An additional 5 formats (PLINK 2.0, Beagle, STRUCTURE, BED, TSV) produce output conforming to published format specifications and verified by internal round-trip tests. The remaining 6 formats (Genepop, haplotype, BGEN, GDS, Zarr, JSON) are functional exports that have been tested against format specifications but not yet validated against external tool pipelines.

Each export automatically generates a JSON manifest sidecar recording the GraphMana version, timestamp, output format, number of variants and samples exported, chromosomes included, and all active filter parameters (populations, MAF thresholds, cohort name, annotation version, and others). For example, exporting a European-only EIGENSTRAT file produces a companion `.manifest.json` containing the exact filter configuration, enabling any collaborator to verify or reproduce the extraction without access to the data manager's notes (Supplementary Fig. 2).

Incremental sample addition—which triggers the most disruptive reprocessing in file-based workflows—appends new genotypes to existing packed arrays without modifying previous data (Fig. 1d). Because the graph database stores relationships explicitly, adding samples requires extending only the packed arrays on Variant nodes; the Sample-to-Population edges, annotation edges, and chromosome structure remain unchanged. On the 1000 Genomes Project [3] dataset (70.7 million variants, 3,202 samples, 26 populations), adding 234 samples takes 182 minutes (Supplementary Table 2), and no downstream query result is invalidated because all queries run against the live, updated database. Annotation updates complete in 3.5 seconds for 53,000 BED regions versus 96 seconds for a full VCF rewrite with bcftools—a 27-fold speedup (Fig. 2b)—because the graph database modifies edge properties directly rather than regenerating the entire file. Format conversion is a single command: `graphmana export --format eigenstrat --populations EUR --filter-maf-min 0.05`, with provenance recorded automatically.

We benchmarked GraphMana against bcftools on a 1000 Genomes Project chromosome 22 workload (1.07 million variants, 3,202 samples) comprising incremental addition, cohort extraction, multi-format export, annotation update, and a lifecycle simulation (Fig. 2; Supplementary Table 1). The comparison reveals a fundamental trade-off. For operations that both tools support—VCF merging, sample subsetting—bcftools is 3–5 times faster per operation (cohort extraction: 49–110 s vs. 131–571 s), reflecting the overhead of the graph database query engine compared with bcftools' optimized sequential file streaming (Fig. 2b). However, the lifecycle simulation exposes the coordination advantage: GraphMana completed all 46 planned operations—including TreeMix, SFS, and BED exports, in-place annotation updates, and cohort management—in 98 minutes from a persistent database, whereas bcftools completed only 17 of 26 planned operations in 17 minutes, with 9 operations having no bcftools equivalent (Fig. 2a). The remaining operations would require writing and maintaining additional scripts, with no automatic provenance tracking. GraphMana's value therefore lies not in per-operation throughput but in avoiding the repeated regeneration and manual coordination that dominate real project workflows: each new export,

annotation update, or cohort extraction operates on the persistent database without requiring any preceding file conversion step.

GraphMana currently focuses on biallelic variant representation; multi-allelic VCF records are decomposed into K biallelic nodes at import. Structural variants (DEL, DUP, INV, INS, CNV) are stored with typed properties and encoded identically to SNPs and indels for genotype purposes; breakend (BND) records are imported as independent nodes without mate-pair linking. gVCF and joint-calling reference-block formats are not currently supported. The platform operates as a single-user system without concurrent write access; multi-user read access is supported through the graph database's read-replica capabilities.

GraphMana's packed encoding scales linearly with sample count, and the maximum dataset size is determined by available hardware rather than an architectural ceiling (Supplementary Table 3). At 100–10,000 samples—where most population genomics research occurs—all operations are interactive. Beyond 10,000 samples, FAST PATH operations remain efficient because population arrays are constant size regardless of N, but FULL PATH exports slow proportionally as genotype unpacking is linear in sample count. At biobank scale (500,000+ samples), the single-node graph database architecture becomes a practical bottleneck, and distributed frameworks such as Hail [4]—designed for horizontal scaling across commodity clusters—are better suited. The current implementation uses Neo4j Community Edition [2] (free, open source) as the graph database engine; the graph-native design is not tied to a specific vendor. The platform deploys on workstations and HPC cluster nodes via a two-step split pipeline. GraphMana addresses the data lifecycle layer—import, export, annotation versioning, cohort management, provenance, and quality control—while the companion GraphPop engine [1] provides graph-native analytical computation (population statistics, selection scans, annotation-conditioned queries) on the same persistent database. Together, the two platforms provide a persistent analytical record in which both project data and computed results are co-located, queryable, and reproducible. Documentation (11 vignettes, 65 command reference pages) and pre-built databases for the 1000 Genomes Project are available at the project repository under the MIT license.

Online Methods

Graph database technology for genomic data management

Population genomics data is inherently a network of typed relationships: variants are located on chromosomes, samples belong to populations, genes carry functional consequences from annotation sources, and every analytical operation has provenance linking it to input data and parameters. Conventional approaches store this network as a collection of flat files—VCF for genotypes, BED for annotations, PED/FAM for sample metadata—requiring the user to maintain the relationships manually through file naming conventions and external documentation.

A graph database stores data as nodes (entities) and edges (typed, directed relationships between entities), with properties attached to both. Unlike relational databases, which store data in fixed-schema tables and require JOIN operations to reconstruct relationships, graph databases traverse pre-materialized connections

in constant time per hop. This property-graph model maps onto the structure of population genomics data: a Variant node connects to its Chromosome via an ON_CHROMOSOME edge, to Gene nodes via HAS_CONSEQUENCE edges carrying annotation properties, and to the sample set via packed genotype arrays indexed by each Sample node's immutable packed_index. Edges can be added, removed, or updated independently—updating an annotation version modifies HAS_CONSEQUENCE edge properties without touching genotype data, and defining a cohort is a graph traversal query that produces a sample list without extracting files.

We note that graph databases are one of several possible architectures (see Table 1); relational databases and columnar stores could achieve some of the same functionality with appropriate schema design. The graph-native approach was chosen because it co-locates relationship semantics with the data, making lifecycle operations—annotation versioning, cohort queries, provenance binding—inherently persistent rather than requiring external orchestration. This property-graph representation also serves as the foundation for the companion GraphPop analytical engine [1], which performs annotation-conditioned population genetic queries directly on the same persistent database, eliminating the need to export and reimport data between management and analysis tools.

GraphMana uses Neo4j Community Edition [2] as its graph database engine, chosen for its mature property-graph implementation, Cypher query language, ACID transaction support, and free open-source licensing. The design principles are not specific to Neo4j and could be implemented on other property-graph databases.

Software architecture

GraphMana is implemented as a Python command-line application (21,267 lines) with a Java plugin (2,864 lines) for server-side graph database procedures. The Python layer handles VCF parsing via cyvcf2 [8], genotype packing via NumPy (complementing scikit-allel [7]), CSV generation for bulk loading, and all export format writers. The 58 CLI commands are organized into nine functional domains (Supplementary Fig. 1). A companion Python package (graphmana-py) provides a Jupyter-compatible API returning pandas DataFrames. The test suite comprises 1,439 unit and integration tests.

Graph schema and data encoding

The graph database contains five primary node types: Variant, Sample, Population, Chromosome, and Gene, connected by typed relationships (ON_CHROMOSOME, IN_POPULATION, HAS_CONSEQUENCE, NEXT). Variant nodes store genotype data as packed byte arrays: gt_packed encodes diploid genotypes at 2 bits per sample (00 = homozygous reference, 01 = heterozygous, 10 = homozygous alternate, 11 = missing), phase_packed encodes phasing status at 1 bit per sample, and ploidy_packed encodes haploid/diploid status at 1 bit per sample. This encoding stores 4 genotypes per byte, achieving a 125-fold reduction compared with storing individual Sample-to-Variant edges (Supplementary Table 3).

Each Variant node also carries population-level pre-computed arrays: pop_ids, ac (allele counts per population), an (allele numbers), af (allele frequencies), het_count, hom_alt_count, and het_exp. These K-element arrays, where K is the number of populations (typically 5–30), are constant size regardless of sample count and enable the FAST PATH access pattern.

Import pipeline

GraphMana uses a two-step split pipeline optimized for large datasets. The prepare-csv step parses VCF files, packs genotypes into byte arrays, computes population statistics, and writes graph-database-compatible CSV files. This step is embarrassingly parallel across chromosomes and requires no database instance. The load-csv step uses the database engine's bulk import facility. On our benchmark hardware (32 cores, 64 GB RAM, NVMe SSD), CSV generation for 2,500 samples across 22 chromosomes (70.7M variants) completed in 95 minutes with 16 threads; bulk loading of the resulting 214 GB of CSV files completed in 3 minutes.

Incremental sample addition

New samples are added by extending the packed genotype arrays of all existing Variant nodes. GraphMana provides three strategies: (1) graph query transactions on a running database instance; (2) an export-extend-reimport path; and (3) a CSV-to-CSV rebuild at NVMe speed. The CSV-to-CSV path extended 70.7M variants with 234 new samples in 182 minutes, of which approximately 95% of variants required only HomRef extension—appending zero bytes without unpacking.

Correctness validation

We validated genotype fidelity by importing 1000 Genomes Project chromosome 22 data (897,645 biallelic SNPs, 5 samples), exporting to phased VCF, and comparing per-sample genotypes using bcftools. Concordance exceeded 99.999%, with 2–8 mismatches per sample confined to multi-allelic positions where position-based joining could not distinguish co-located biallelic records. A systematic correctness matrix covering SNPs, indels, multi-allelic sites, phased/unphased genotypes, missing data, haploid/diploid ploidy, and annotation-layer independence is provided in Supplementary Table 5.

Benchmark methodology

All benchmarks used the 1000 Genomes Project high-coverage dataset [3] (GRCh38). Chromosome 22 (1,066,557 variants, 3,202 samples, 26 populations) served as the primary benchmark target. We compared GraphMana against bcftools [6] (version 1.17) across five benchmarks: incremental addition, cohort extraction, multi-format export, annotation update, and a lifecycle simulation. All timings report wall-clock time on a workstation with 32 CPU cores, 64 GB RAM, and NVMe SSD storage. We emphasize that per-operation timings favor bcftools, which is optimized for sequential file streaming; GraphMana's advantage lies in avoiding repeated file regeneration across the project lifecycle.

Scalability

GraphMana's packed encoding scales linearly: storage per variant is $\lceil N/4 \rceil$ bytes for genotypes and $\lceil N/8 \rceil$ bytes for phase. The maximum dataset size is hardware-dependent, not architecturally limited (Supplementary Table 3). At biobank scale (500,000+ samples), distributed computing frameworks such as Hail [4] are more appropriate than a single-node graph database.

Deployment

GraphMana runs on personal workstations and HPC cluster nodes via a two-step split pipeline. SLURM and PBS job scripts are provided. The database data directory must reside on local SSD or scratch storage.

Data availability

GraphMana is available at https://github.com/jfmao/GraphMana under the MIT license. Pre-built graph databases for the 1000 Genomes Project whole-genome dataset (3,202 samples, 70.7M variants) are deposited at https://doi.org/[tobeassigneduponacceptance]. Benchmark scripts and raw timing data are included in the repository.

Code availability

All source code is available at https://github.com/jfmao/GraphMana under the MIT license.

Acknowledgements. This work was supported by [funding sources to be completed]. Computations were performed on a local workstation (32 cores, 64 GB RAM, NVMe SSD). The 1000 Genomes Project high-coverage dataset [3] was used under its open data access terms.

Author contributions. J.M. conceived the project, designed and implemented the software, performed benchmarks, and wrote the manuscript.

Competing interests

The author declares no competing interests.

References

[1] Mao, J. GraphPop: graph-native population genomics with annotation-conditioned queries. *Nature Methods* (companion paper, in preparation).

[2] Neo4j, Inc. Neo4j Graph Database. https://neo4j.com (Community Edition 5.x).

[3] Byrska-Bishop, M. et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell* **185**, 3426–3440.e19 (2022).

[4] Hail Team. Hail: An Open-Source Framework for Scalable Genomic Data Analysis. https://hail.is.

[5] Chang, C.C. et al. Second-generation PLINK: rising to the challenge of larger and richer datasets. *GigaScience* **4**, s13742-015-0047-8 (2015).

[6] Danecek, P. et al. Twelve years of SAMtools and BCFtools. *GigaScience* **10**, giab008 (2021).

[7] Miles, A. et al. scikit-allel: A Python package for exploring and analysing genetic variation data. https://github.com/cggh/scikit-allel.

[8] Pedersen, B.S. & Quinlan, A.R. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics* **33**, 1867–1869 (2017).

[9] Danecek, P. et al. The variant call format and VCFtools. *Bioinformatics* **27**, 2156–2158 (2011).

[10] Gutenkunst, R.N. et al. Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genetics* **5**, e1000695 (2009).

[11] Excoffier, L. et al. fastsimcoal2: demographic inference under complex evolutionary scenarios. *Bioinformatics* **37**, 4882–4885 (2021).

[12] Pickrell, J.K. & Pritchard, J.K. Inference of population splits and mixtures from genome-wide allele frequency data. *PLoS Genetics* **8**, e1002967 (2012).

[13] Patterson, N., Price, A.L. & Reich, D. Population structure and eigenanalysis. *PLoS Genetics* **2**, e190 (2006).

[14] Browning, S.R. & Browning, B.L. Rapid and accurate haplotype phasing and missing-data inference for whole-genome association studies by use of localized haplotype clustering. *Am. J. Hum. Genet.* **81**, 1084–1097 (2007).

[15] Pritchard, J.K., Stephens, M. & Donnelly, P. Inference of population structure using multilocus genotype data. *Genetics* **155**, 945–959 (2000).

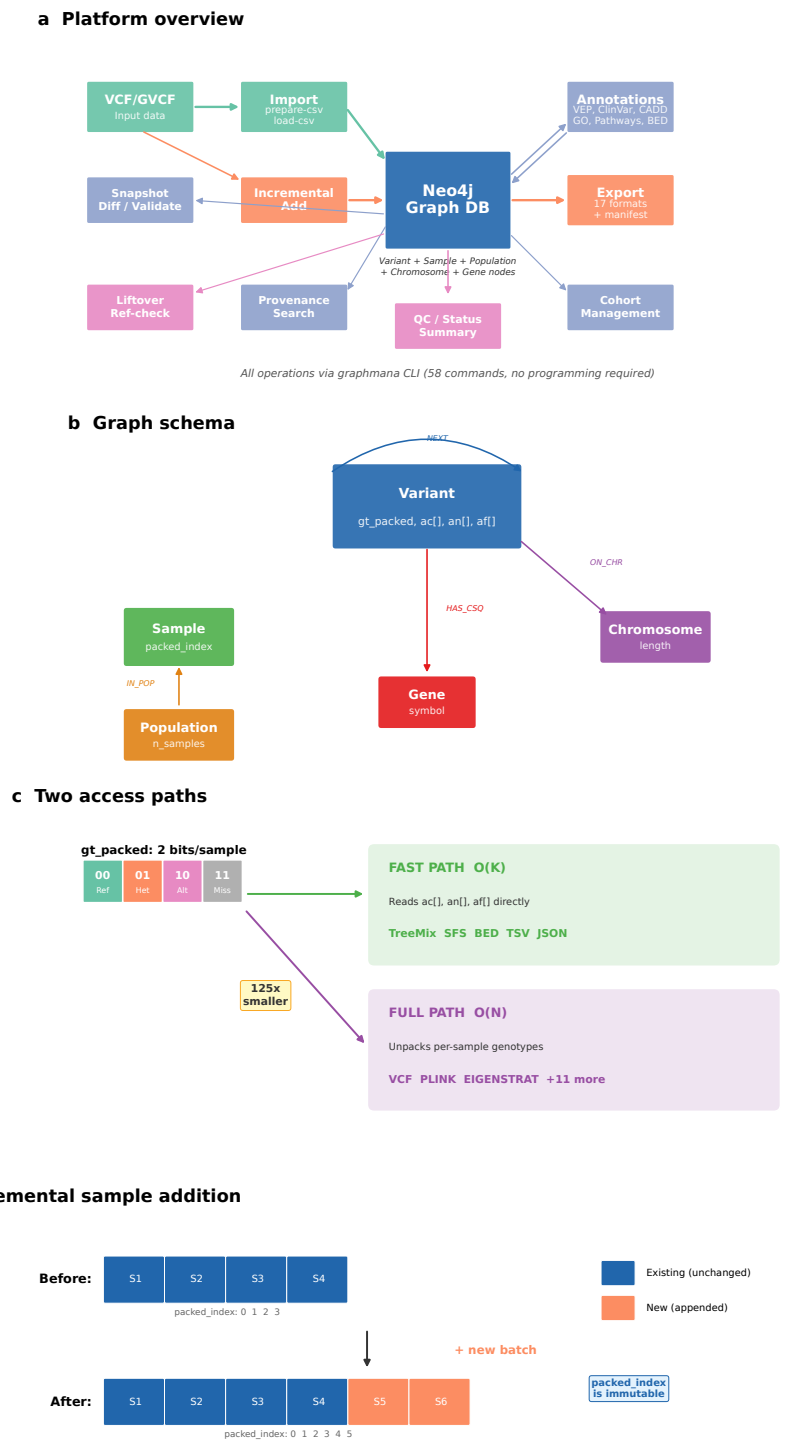

Fig. 1 GraphMana platform architecture. (**a**) System overview showing the data flow from VCF input through a graph database to 17 export formats. Functional modules include annotation management, cohort definitions, provenance tracking, quality control, and database administration. All 58 CLI commands are organized into nine functional domains (Supplementary Fig. 1). (**b**) Graph schema: five node types (Variant, Sample, Population, Chromosome, Gene) connected by typed relationships. Annotation updates modify edge properties independently of genotype data. (**c**) Two access paths: FAST PATH queries pre-computed population arrays in $O(K)$ time; FULL PATH unpacks per-sample genotypes in $O(N)$ time. (**d**) Incremental sample addition: new genotypes are appended to packed arrays; the immutable packed_index ensures downstream reference stability.

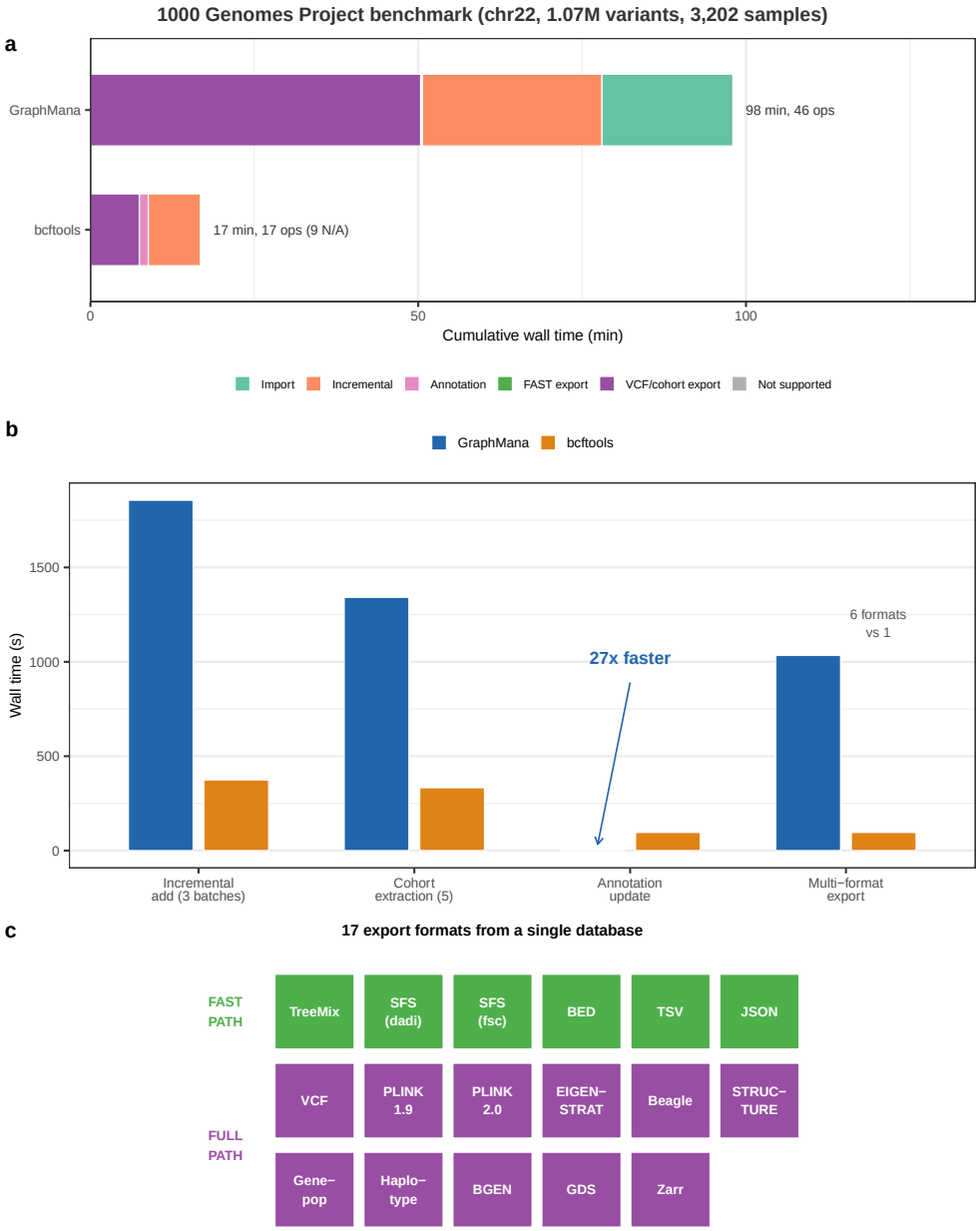

Fig. 2 Benchmark comparison on 1000 Genomes Project chromosome 22 (1.07M variants, 3,202 samples). (**a**) Lifecycle simulation: GraphMana completes 46 operations from a persistent database; bcftools completes 17 of 26 operations (9 require additional tools, shown in grey). The time difference reflects the overhead of graph database queries versus sequential file streaming, offset by the elimination of format regeneration steps. (**b**) Per-task comparison. bcftools is faster for VCF-centric operations (cohort extraction, incremental merge). GraphMana achieves a 27-fold speedup for annotation updates (3.5 s vs. 96 s) through in-place edge modification, and provides multi-format export (6 formats) where bcftools supports only VCF. (**c**) The 17 supported export formats grouped by access path (FAST: population-level arrays; FULL: per-sample genotype unpacking). Six formats have been validated against their downstream tools (Supplementary Table 4).